

TRANSITAI

AI-Based Smart College Bus System

Overcrowding Detection & Route Optimization



FINAL COMPLETION / PROTOTYPE DOCUMENTATION

This document upgrades the uploaded 35% project concept into a complete demonstrable prototype design. It preserves the original flow: demand prediction, overcrowding prediction, ETA analysis and bus-allocation recommendation.

Project	AI-Based Smart College Bus System
Users	Students + College Transport Management
Core Output	Demand prediction + crowd level + recommended action
Data	Historical count, time, day, events, route, GPS, traffic, occupancy
Prototype Status	End-to-end demonstrable workflow; live integrations are ready for later connection

1. Problem & Objective

College buses may run on fixed routes and schedules even when passenger demand changes. The original project identifies overcrowded buses, unused seats, unpredictable waiting time, traffic-driven arrival changes and poor demand visibility as the key problems.

Objective: build a smart system that predicts passenger demand, detects overcrowding, estimates arrival conditions and recommends efficient bus deployment.

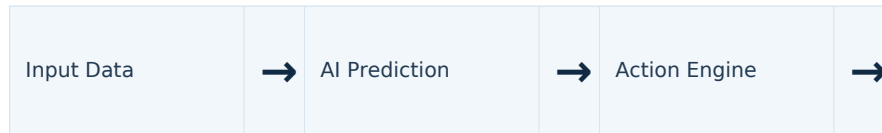
Problem	Overcrowding on some routes while other buses may have spare capacity.
AI Role	Learn recurring demand patterns and predict expected passenger demand.
Decision Role	Recommend allocation/route actions before overcrowding becomes severe.
Student Value	Visibility of crowd level, ETA and route status.

2. Complete System Modules

Student Module	Live bus status, crowd level, ETA and route information.
Bus/GPS Data	Route, bus ID, current occupancy, location and operating status.
Data Processing	Validation, normalization and preparation of transport data.
AI Demand Prediction	Expected passenger demand for each route and time slot.
Crowd Classification	Low / Medium / High crowd-risk classification.
ETA Analysis	Travel-time and traffic conditions used for arrival estimation.
Optimization Engine	Compares demand, capacity, available buses, travel time and waiting time.
Admin Dashboard	Fleet view, alerts, analytics and AI recommendations.
History	Stores predictions and observed outcomes for future model improvement.



3. End-to-End Workflow



1. Collect

Inputs include historical passenger count, time, day, exam/college events, route, GPS/location, traffic and current occupancy.

2. Predict

The AI model estimates expected passenger demand for the selected route and time slot.

3. Classify

Predicted demand is compared with effective capacity and converted to a Low, Medium or High crowd-risk state.

4. Decide

The system evaluates available buses, expected waiting time, distance/travel time and traffic before recommending an action.

5. Learn

Actual occupancy and outcomes can be stored to improve future training and evaluation.

4. AI Prediction & Crowd Logic

The uploaded project deck defines historical passenger count, time, day, exam/college events, route, GPS, traffic and current occupancy as the main AI inputs.

Time / Day	Captures recurring peak-hour and weekly demand patterns.
Current Occupancy	Shows the current pressure on the bus.
Historical Count	Provides route-specific demand history.
Events	Exams and campus events can change demand.
Traffic	Changes travel time and arrival conditions.
Route	Different routes have different demand profiles.

Prototype decision formula

Crowd Ratio = Predicted Demand ÷ Effective Capacity

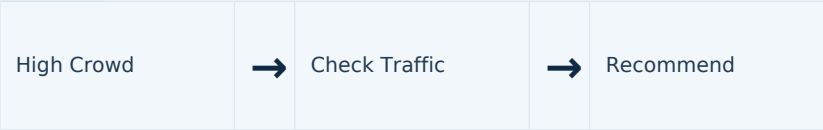
Suggested prototype classes: LOW when demand is comfortably below capacity; MEDIUM when approaching capacity; HIGH when demand reaches or exceeds the operational capacity threshold.

Important: the uploaded source does not provide a validated real-campus accuracy percentage or a fixed production ML algorithm. This final document therefore does not invent an accuracy result.

5. Smart Route Optimization

The original project compares passenger demand, distance/travel time, traffic conditions, bus capacity, number of available buses and expected waiting time.

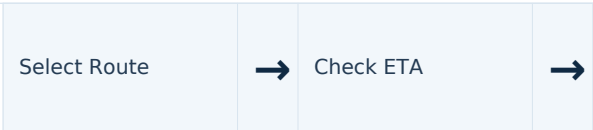
Demand	Expected passengers on each route/time slot.
Capacity	Effective passenger capacity of candidate buses.
Availability	Which buses can be reassigned without creating another shortage.
Travel Time	How quickly a candidate bus can reach the affected route.
Traffic	Expected congestion/delay.
Waiting Time	Student-facing impact of under-served routes.



The system recommends decisions; transport staff remain in control. This keeps the prototype advisory rather than claiming fully autonomous fleet control.

6. Student Dashboard - Final Design

Route Card	Selected route, direction and bus status.
Crowd Card	Predicted Low / Medium / High crowd level.
ETA Card	Estimated arrival status and traffic condition.
Stops	Upcoming stops and route information.
Alternative	Recommended alternative bus/route when high crowd risk is detected.
Freshness	Clearly label live, recently updated or simulated data.



For the demonstration, simulated transport data can drive the same interface. GPS, Maps and traffic APIs are integration points for a later live deployment.

7. Admin Dashboard - Final Design

Fleet Overview	Active buses, routes and available buses.			
Crowd Heat View	Routes ranked by current/predicted crowd pressure.			
Prediction Panel	Expected demand for upcoming time slots.			
Alerts	Routes predicted to approach/exceed capacity.			
Recommendation Panel	Suggested bus allocation or route action.			
Analytics	Occupancy history and prediction-vs-observation review.			

Route A HIGH

→

Traffic OK

→

Notify

The key value is the conversion of AI prediction into an operational recommendation instead of only displaying a crowd number.

8. Backend & Data Design

The uploaded deck lists React/React Native, Node.js + Express or FastAPI, Python/Pandas/NumPy/scikit-learn or TensorFlow, MongoDB/Firebase and GPS/Maps/traffic data as the technology stack.

Occupancy Record	route_id, bus_id, timestamp, occupancy, capacity, location, traffic, event
Prediction Record	route_id, prediction_time, predicted_demand, crowd_level
Recommendation	affected_route, candidate_bus, action, waiting_time, reason
Feedback	actual_occupancy, actual_arrival, action_result

API contract

```

POST /api/occupancy
POST /api/predictions
GET  /api/routes/:routeId/status
GET  /api/admin/alerts
POST /api/recommendations/acknowledge

```

9. AI Prototype Code

A compact baseline implementation for the demand-prediction layer:

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor

features = [
    "hour", "day_of_week", "current_occupancy",
    "historical_avg", "event_flag", "traffic_level"
]

df = pd.read_csv("bus_history.csv")
X, y = df[features], df["passenger_count"]

model = RandomForestRegressor(
    n_estimators=120, random_state=42
)
model.fit(X, y)

def predict_demand(record, capacity):
    x = pd.DataFrame([record])[features]
    demand = float(model.predict(x)[0])
    ratio = demand / max(capacity, 1)

    if ratio < 0.60:
        level = "LOW"
    elif ratio < 0.85:
        level = "MEDIUM"
    else:
        level = "HIGH"

    return {
        "predicted_demand": round(demand, 1),
        "crowd_level": level,
        "capacity_ratio": round(ratio, 2)
    }
```

This is prototype logic. A real campus deployment must train and validate the model using actual college-bus observations.

10. Recommendation Engine

```
def recommend_action(prediction, available_buses,
                    current_wait_minutes, traffic_level):

    level = prediction["crowd_level"]

    if level == "LOW":
        return {"action": "KEEP_CURRENT_ALLOCATION",
                "reason": "Capacity is sufficient."}

    if level == "MEDIUM":
        return {"action": "MONITOR_AND_PREPARE",
                "reason": "Demand is approaching capacity."}

    if available_buses > 0 and traffic_level != "HEAVY":
        return {"action": "DEPLOY_AVAILABLE_BUS",
                "reason": "High demand and suitable bus available."}

    if current_wait_minutes > 15:
        return {"action": "PRIORITIZE_ALTERNATIVE_ROUTE",
                "reason": "High crowd risk and long waiting time."}

    return {"action": "ALERT_TRANSPORT_STAFF",
            "reason": "High crowd risk needs manual decision."}
```

Example: if Route A is predicted HIGH, another bus is available, and traffic is acceptable, the system recommends deploying that bus. If no suitable bus exists, it raises an alert instead of pretending that the problem has been solved.

11. Testing & Validation

T01	Low demand → LOW + keep allocation.
T02	Demand approaching capacity → MEDIUM + monitor.
T03	Demand above capacity → HIGH + action recommendation.
T04	HIGH + available bus + acceptable traffic → deploy bus.
T05	HIGH + no suitable bus → alert / alternative route.
T06	Heavy traffic → ETA/decision reflects traffic condition.
T07	Invalid occupancy → input rejected or flagged.

Recommended real-world evaluation metrics

Demand prediction: MAE/RMSE. Crowd classification: precision, recall, F1-score and confusion matrix. ETA: mean absolute error in minutes. Operational impact: waiting-time reduction, overcrowding incidents and bus utilization.

No invented accuracy numbers are included because the uploaded source does not provide validated real-campus results.

12. Final Demo Scenario

1. Start	Open admin dashboard and show three routes with different occupancy levels.
2. Trigger	Set Route A to high occupancy during a peak-hour time.
3. Predict	Run the AI prediction and display HIGH crowd risk.
4. Analyze	Show that another bus is available and traffic conditions are acceptable.
5. Recommend	System recommends deploying the available bus to Route A.
6. Student View	Open student screen and show updated crowd status/ETA.
7. Feedback	Record the action and show it in the admin activity/history.

Demo statement

“TransitAI does not simply detect a crowded bus. It predicts demand, evaluates the operational context and converts the prediction into an actionable transport recommendation.”

13. Security, Privacy & Reliability

Access Control	Separate student and transport-admin permissions.
Validation	Reject impossible occupancy, invalid timestamps and malformed route data.
Privacy	If camera counting is added, avoid unnecessary personal-image storage and apply appropriate privacy controls.
Network Failure	Show last known data with a stale-data indicator instead of presenting it as live.
AI Uncertainty	Predictions are recommendations, not guarantees.
Audit Trail	Store recommendation time, route, action and outcome for review.

These points address the original project's identified challenges: accurate passenger data, GPS/network failures, unexpected traffic and privacy concerns for camera-based counting.

14. Final Completion Status & Future Scope

Problem & Objective	Completed
AI Workflow	Prototype implemented
Crowd Classification	Capacity-aware logic implemented
Recommendation Engine	Prototype implemented
Student UI	Final module specification
Admin UI	Final module specification
Data/API Contract	Defined
Testing Plan	Defined
Live GPS/Traffic	Integration-ready; needs actual service/data
Real Campus Accuracy	Requires actual college-bus dataset

Future scope

Dynamic route changes during special events, predictive maintenance, personalized bus recommendations, EV charging/energy optimization, digital passes with automated occupancy data, smart-campus integration and edge AI for real-time passenger counting.

CONCLUSION

TransitAI moves college transport from a fixed, reactive model toward a predictive workflow: DETECT → PREDICT → OPTIMIZE → ACT. It combines demand, occupancy, traffic and fleet availability to support better bus allocation and reduce overcrowding risk.

FINAL PROTOTYPE DOCUMENT • END